

Uke 4 – 6 Notater:

IN2090 – Databaser og datamodellering

Du er på uke uke 40:

Funksjonell avhengighet:

- Et attributt A er funksjonelt avhengig av en mengde attributter X hvis det bare finnes en verdi av A for hver mengde verdier av attributtene i X.
- Det skrives $X \rightarrow A$, og en slik formel kalles en funksjonell avhengighet (FD).
- For eksempel er KARAKTER funksjonelt avhengig av {BRNAVN, KURSKODE} i karakter-tabellen:

Karakter		
Brnavn	Kurskode	Kara
evgenit	IN2090	B
peternl	IN2090	A
evgenit	IN2080	B
leifhka	IN2090	B
leifhka	IN3110	C

FDer, data og virkeligheten:

- FDer uttrykker det vi mener er sant i virkeligheten som dataene våre beskriver
- f.eks er brukernavnet til en student faktisk unikt for hver student, mens adressen kanskje ikke trenger å være det.
- Kan fort bli et komplisert spørsmål om verdens tilstand
- FDer forteller oss hvilke data hører sammen, og hva de hører til

Syntaks for FDer:

- Leser ofte pilen som «bestemmer», så
 $X \rightarrow Y$
leses som «X bestemmer Y» eller «Y er funksjonelt avhengig av X».
- Vi dropper ofte mengde-tegnene i FDer, så istedenfor å skrive:
 $\{BRNAVN, KURSKODE\} \rightarrow \{KARAKTER\}$
Skriver vi ofte:
 $BRUKERNAVN, KURSKODE \rightarrow KARAKTER$

- Dersom attributtene er enkle bokstaver (A, B, osv) dropper vi ofte også komma og skriver istedenfor dette:
 $A, B \rightarrow X, Y, Z$
 Skriver vi heller:
 $AB \rightarrow XYZ$

FDers oppførsel:

- Vi kan samle opp høyresider i FDer, og skrive:
 $X \rightarrow A, B$
 Dersom vi både har $X \rightarrow A$ og $X \rightarrow B$.
- FDer er transitive: hvis $X \rightarrow Y$ og $Y \rightarrow Z$, så har vi at $X \rightarrow Z$.
- En FD $X \rightarrow Y$ hvor Y (delmengde av) X kalles triviell, altså hvis Y er funksjonelt avhengig av X og der Y er en delmengde av X så er det trivielt:
 $BRNAVN, NAVN \rightarrow NAVN$
- Vi ignorerer slike trivielle FDer, fordi de alltid er sanne og dermed ikke gir oss noe informasjon

Eksempel:

$R(\text{Brnavn}, \text{Navn}, \text{Etternavn}, \text{Adresse}, \text{Kurskode}, \text{Tittel}, \text{Beskrivelse}, \text{AntSP}, \text{Karakter})$

Foreslår følgende FDer:

- $\text{Brnavn} \rightarrow \text{navn}, \text{Etternavn}, \text{Adresse}$
- $\text{Kurskode} \rightarrow \text{Tittel}, \text{Beskrivelse}, \text{AntSP}$
- $\text{Brnavn}, \text{Kurskode} \rightarrow \text{Karakter}$

Vi skal nå se på forholdet mellom funksjonelle avhengigheter og nøkler:

Nøkler:

- En supernøkkel for en relasjon er jo enhver mengde attributter som sammen er unike for relasjonen.
- En kandidatnøkkel er en (delmengde)-minimal supernøkkel
- Dersom en mengde attributter er unike forekommer jo hver kombinasjon av disse kun i et tuppel, og bestemmer derfor de andre verdiene i tuplet
- Med andre ord, en nøkkel (enten super eller kandidat) er en mengde attributter.
- Altså, Fdene sier hvilke supernøkler og kandidatnøkler vi har!

FDer og nøkler:

- Dersom R er en relasjon med attributter X, så vil:
 - Y (delmengde av) X være en supernøkkel for R hvis $Y \rightarrow X$ / Y (Y bestemmer X, der Y ikke er en delmengde av X), som er ekvivalent med $Y \rightarrow X$
 - Y (delmengde av) X er en kandidatnøkkel for R hvis Y er en minimal supernøkkel.
- For å sjekke om X er en supernøkkel, sjekk om alt er avhengig av X.
- Altså, bruk FDe og finn alle attributter som er avhengige av X, de som er avhengige av disse igjen, osv.

Tillukning:

- Tillukning X^{+} av X på en mengde FDer er mengden attributter som er funksjonelt avhengige av X
- Hvis $X \rightarrow A$, sp er A (element) X^{+} sant
- Hvis A (ikke element) X^{+} , så er ikke $X \rightarrow A$ sant
- Tillukningen kan regnes ut ved å bruke FDe om og om igjen:
 - Sett $X^{+} = X$
 - Så lenge X^{+} forandres:
 - Finn en FD $Y \rightarrow Z$ med Y (delmengde av) X^{+}
 - Sett $X^{+} = X^{+} \cup (\text{union}) Z$.

Eksempel Tillukning:

Gitt følgende FDer:

- Brnavn $\rightarrow$ Navn, Adresse
- Kurskode $\rightarrow$ Grad
- Brnavn, Kurskode $\rightarrow$ karakter
- Grad, Karakter $\rightarrow$ Bestått

Så har vi følgende tillukninger:

- $\text{Brnavn}^{+} = \text{Brnavn, Navn, Adresse}$ _____ Grunnen til at det ble slik er fordi Brnavn som er alene på FD-siden kun har Navn og Adresse på høyresiden.
- $\{\text{Brnavn, Kurskode}\}^{+} = \text{Brnavn, Kurskode, Navn, Adresse, Grad, Karakter, Bestått}$ _____ Grunnen til at Brnavn og Kurskode har de tre over er fordi det er det Brnavn attributten

har. Kurskode har Grad og Karakter. Siden vi har Grad og karakter så har vi også Bestått.

- $\text{Navn}^+ = \text{Navn}$ _____ Navn har kun seg selv på venstresiden, dermed er det alt.
- $\text{Grad}^+ = \text{Grad}$ _____ Grad har kun seg selv siden vi ikke har en enslig Grad på FD-siden(venstresiden)

Finne kandidatnøkler:

- Vi må sjekke alle delmengder av attributter, nedenfra. Men, følgende to regler hjelper oss:
 - Hvis A ikke forekommer i noen høyreside, er A med i ALLE kandidatnøkler.
 - Hvis A forekommer i minst en høyreside, men ingen venstresider, er A IKKE DEL av noen kandidatnøkkel.
- Så begynn med alle attributter som ikke forekommer på høyre side. Beregn tillukningen.
- Hvis alle attributter er med, sjekk minimalitet. Hvis ikke, utvid i tur og orden med ett og ett nytt attributt.

Eksempel (lett):

$R(\text{Brnavn}, \text{Navn}, \text{Etternavn}, \text{Adresse}, \text{Kurskode}, \text{Tittel}, \text{Beskrivelse}, \text{AntSP}, \text{Karakter})$

- $\text{Brnavn} \rightarrow \text{Navn}, \text{Etternavn}, \text{Adresse}$
- $\text{Kurskode} \rightarrow \text{Tittel}, \text{Beskrivelse}, \text{AntSP}$
- $\text{Brnavn}, \text{Kurskode} \rightarrow \text{Karakter}$

Starter med å finne alle attributter som ikke er på høyre side: Brnavn, Kurskode.

Så finner vi attributter som er i høyresider, men ikke i venstresider: Navn, Etternavn, Adresse, Tittel, Beskrivelse, AntSP, Karakter

Det vil si at $\{\text{Brnavn}, \text{Kurskode}\}$ er den eneste kandidatnøkkelen.

Eksempel (litt mer komplisert):

$R(\text{Brnavn}, \text{Navn}, \text{Etternavn}, \text{Adresse}, \text{Kurskode}, \text{Tittel}, \text{Beskrivelse}, \text{AntSP}, \text{Karakter}, \text{Bestått})$

- $\text{Brnavn} \rightarrow \text{Navn}, \text{Etternavn}, \text{Adresse}$
- $\text{Kurskode} \rightarrow \text{Tittel}, \text{Beskrivelse}, \text{AntSP}$

- Tittel → Kurskode, Beskrivelse, AntSP
- Brnavn, Kurskode → Karakter
- Karakter → Bestått

Starter med å finne alle attributter som ikke er på høyre side: Brnavn

Så finner vi attributter som er i høyresider, men ikke i venstresider: Navn, Adresse, Beskrivelse, AntSP, Bestått

Siden Brnavn alene ikke gjør rede for absolutt alle attributtene i databasen R, så er det ikke en kandidatnøkkel.

Det vil si at kandidatnøkkel er noe med Brnavn + Kurskode/Tittel/Karakter

$X = \text{Brnavn, Kurskode} \xrightarrow{+} X^+ = \text{Brnavn, Kurskode, Navn, Adresse, Tittel, beskrivelse, AntSP, Karakter, Bestått}$

Altså er kandidatnøkklene: {Brnavn, Kurskode} og {Brnavn, Tittel}

Oppsummering så langt:

- Skjemaer som er dårlig designet inneholder anomalier
- Som regel skyldes dette at ikke-relatert informasjon er i samme tabell
- FDer sier hvilken informasjon som henger sammen, samt hvilke nøkler tabeller har.
- Der og nøkler gir oss dermed det vi trenger for å spesifisere kriterier for når vi får anomalier og ikke.
- Disse kriteriene definerer ulike normalformer.

Normalformer:

Normalformene 1NF-BCNF:

- Normalformene vi skal se på danner et hierarki:
BCNF (delmengde av) 3NF (delmengde av) 2NF (delmengde av) 1NF
- Høyere NF gir færre anomalier, men flere tabeller og flere joins
- Gitt et skjema, så finnes det en algoritme som lager et ekvivalent skjema på hvilken NF man ønsker.
- For ALLE NF er det slik at et skjema oppfyller en gitt NF hvis alle tabeller oppfyller kravene

1NF (Definisjonen av den første normalformen):

- En tabell er på 1NF hvis alle attributter er atomære
- Altså ikke attributter med arrays/JSON/osv.
- Heller ikke strenger som inneholder flere verdier.
- Av og til ullent begrep (PostgreSQL støtter for eksempel arrays og JSON som datatyper)
- Poenget er at verdiene man ønsker å bruke i joins, sammenlikninger, ol. Skal være i egne kolonner.
- Dette er slik at man skal slippe kompleks manipulering av datastrukturer eller strenger for enkle operasjoner.
- Vi antar derfor at 1NF alltid er oppfylt.

1NF, eksempler på brudd:

Student			
Brnavn	...	Veiledere	Adresse
evgenit	...	[arild, martingi]	Gateveien 1b, 0123 Oslo
peternl	...	[abc]	Stedplassen 2, 1234 Bergen

Ansatt		
Brnavn	...	Studenter
arild	...	[evgenit]
abc	...	[peternl]

Nesten alltid lurene å:

- Lage en egen tabell Veiledning(Student, Veileder)
- Splitte strengen opp i Student(..., Gate, Postnummer, Poststed)

2NF:

- En tabell oppfyller 2NF hvis:
 - Den oppfyller 1NF og
 - Alle attributter A som ikke er nøkkelattributter, IKKE er funksjonelt avhengige av en delmengde av en kandidatnøkkel
- Nøkkelattributt: Attributt som er med i en kandidatnøkkel.
- Alternativt: En tabell BRYTER 2NF hvis det finnes et ikke-nøkkelattributt A som ER AVHENGIG av en delmengde av en kandidatnøkkel.

2NF, eksempel:

Følgende tabell er på 1NF, men ikke 2NF:

R(Brnavn, Navn, Etternavn, Adresse, Kurskode, Tittel, Beskrivelse, AntSP, Karakter)

- Brnavn $\rightarrow$ Navn, Etternavn, Adresse
 - Kurskode $\rightarrow$ Tittel, Beskrivelse, AntSP
 - Brnavn, Kurskode $\rightarrow$ Karakter
-
- kandidatnøkkel: Brnavn, Kurskode
 - Navn er avhengig av brnavn og Brnavn er en del av nøkkelen. Brudd på 2NF.

3NF:

- En tabell oppfyller 3NF hvis:
 - Den oppfyller 2NF og
 - Alle ikke-nøkkelattributter KUN er avhengige av kandidatnøkler
- Alternativt: En tabell bryter 3NF hvis det finnes et ikke-nøkkelattributt som ER AVHENGIG av noe som IKKE er en kandidatnøkkel.
- En tabell på 2NF og ikke 3NF kan kun skje dersom en ikke-nøkkelattributt A er avhengig av en (eller fler) ikke-nøkkelattributt(er) X som selv er avhengig av en kandidatnøkkel.
- Altså, om K er en kandidatnøkkel kunne vi over ha FDene $K \rightarrow X$ og $X \rightarrow A$.
- Altså, A avhenger ikke direkte av kandidatnøkkelen, men transitivt.

3NF, eksempel:

Følgende tabell er på 2NF, men ikke 3NF:

- Ansatt(Id, Navn, Avdeling, Avdelingskode)

Gitt FDene:

- Id $\rightarrow$ navn, Avdelingskode
- Avdelingskode $\rightarrow$ Avdeling

Her har vi:

- Id er en kandidatnøkkel (transitivitet).
- Alle attributter avhenger av hele kandidatnøkkelen, altså Id (2NF)
- Men Avdeling er avhengig av Avdelingskode, som ikke er en nøkkelattributit.
- Avdeling dobbeltlagres for hver ansatt (burde skilles ut i egen tabell).

BCNF:

- Kort for Boyce-Codd normalform
- En tabell oppfyller BCNF hvis alle attributter kun er avhengige av en kandidatnøkkel.
- Samme som 3NF, men unntaket for nøkkelattributter er FJERNET.
- Unntaket blir sjeldent brukt, og som regel er tabeller på 3NF også på BCNF.
- Huskeregel for BCNF: «The key, the whole key, and nothing but the key, (so help me Code)».

BCNF, eksempel:

Følgende tabell er på 3NF, men ikke BCNF:

Nærmeste butikk

Person	Type	Nærmeste
David	Optiker	Ørneøyet
David	Frisør	Kutt og krøll
Kari	Bokhandler	Merlins bøker
Erik	Bakeri	Deiglig
Erik	Frisør	Klipperud

Med følgende FDer:

- Person, Type → Nærmeste
- Nærmeste → Type

Her har vi:

- Kandidatnøkler: {Person, Type} og {Person, Nærmeste}.
- Alle ikke-nøkkelattributter (ingen slike) avhenger kun av kandidatnøkler (altså 3NF).
- Men nøkkel-attributtet Type avhenger av Nærmeste som ikke er en kandidatnøkkel.

Normalformer i praksis:

- Jo høyere normalform, jo færre anomalier.
- Men jo høyere normalform, jo fler relasjoner i skjemaet.
- BCNF fjerner (nesten) alle anomalier og er et godt kompromiss.
- De fleste ordentlige skjemaene (Nortwind, filmdb, osv.) er på BCNF.

Uke 40 Torsdags forelesningsopptak!!!!!!

Eksempler: Databasesdesign og normalformer.

Oppgave 1)

Gitt følgende relasjon:

Person(personnr, navn, initialer, fødselsdato, alder)

Med FDene:

- Personnr $\rightarrow$ navn, fødselsdato
- Navn $\rightarrow$ initialer
- Fødselsdato $\rightarrow$ alder

Finn tillukningene til:

1.1 navn: $\text{navn}^+ = \text{navn}, \text{initialer}$

1.2 personnr: $\text{personnr}^+ = \text{personnr}, \text{navn}, \text{fødselsdato}, \text{initialer}, \text{alder}$

2. Finn alle kandidatnøkklene til Person:

- Aldri på høyresider: personnr
- Bare på høyresider: initialer, alder.
- Forsøke å utvide med: navn, fødselsdato hvis personnr ikke er en kandidatnøkkel, men det er den

Oppgave 2)

Finn kandidatnøkklene til relasjonen:

Produkt(produktID, navn, kategori, pris, butikkID, butikknavn, butikktype, adresse, poster, poststed)

Med FDene:

- produktID $\rightarrow$ navn, kategori, pris
- Navn, kategori $\rightarrow$ produktID
- butikkID $\rightarrow$ butikknavn, butikktype, adresse, postnr
- postnr $\rightarrow$ poststed

Kandidatnøkler:

1. produktID, butikkID
2. Navn, kategori, butikkID

Oppgave 3)

Gitt relasjonen R(A, B, C, D, E, F, G)

Med FDene:

- $AB \rightarrow DE$
- $C \rightarrow A$
- $BD \rightarrow E$
- $AE \rightarrow BF$

Finn alle nøklene:

Kun på venstreside: C, G

Kun på høyreside: F

Kandidatnøkler:

- $\{C, B, G\}^+ = C \rightarrow A, AB \rightarrow DE, BD \rightarrow E, AE \rightarrow BF$
 $\{C, B, G\}^+ = A, B, C, D, E, F$
- $\{C, E, G\}^+ = C \rightarrow A, AE \rightarrow BF, AB \rightarrow DE, BD \rightarrow E$
 $\{C, E, G\}^+ = A, B, C, D, E, F$

Uke 41:

Hvordan sjekke normalformer:

- For å sjekke hvilken NF et skjema oppfyller, kan vi sjekke alle tabeller opp mot alle FDer
- Og deretter sjekke om FDen bryter med BCNF, 3NF og 2NF
- Vi skal nå se en algoritme som gjør dette systematisk
- Det er derimot to ting vi må gjøre med FDene før vi kan bruke algoritmen:
 Skrive om Fder og fjerne redundans.

Skrive om FDer:

- FDene må skrives på formen $X \rightarrow A$ (splitt høyresidene).

- F.eks fremfor:
 ProduktID $\rightarrow$ Navn, Pris
 Må man skrive:
 ProduktID $\rightarrow$ Navn
 ProduktID $\rightarrow$ Pris
- Algoritmen antar altså at det kun er én attributt på høyresiden

Fjerne redundans:

- Hvis $X \rightarrow B$, så har vi også $X \cup Y \rightarrow B$ for alle Y
- F.eks dersom:
 ProduktID $\rightarrow$ Navn
 Så har vi jo også:
 ProduktID, Pris $\rightarrow$ Navn
 men ønsker kun å bruke den øverste.
- Vi må fjerne alle redundante FDer, algoritmen antar ingen redundans
- Må gjøres når man bestemmer FDene

Algoritme for å sjekke NF:

- Først, finn alle kandidatnøkler med algoritmen fra forrige uke
- For hver tabell og hver FD $X \rightarrow A$:
 1. Er x en supernøkkel?
 Ja: BCNF så langt, gå til neste FD
 Nei: Brudd på BCNF. Gå til 2. punkt
 2. Er A et nøkkelattribut?
 Ja: 3NF så langt, gå til neste FD
 Nei: Brudd på 3NF. Gå til 3. punkt
 3. Er X del av en kandidatnøkkel?
 Ja: brudd på 2NF og skjema er på 1NF, stopp.
 Nei: 2NF så langt, gå til neste FD
- Tabellen er så på den laveste normalformen vi får ut av denne algoritmen
- Skjemaet er nå den laveste normalformen av tabellenes.
- Med andre ord: Hvis jeg har en tabell og en FD som bryter 2NF, er skjemaet på 1NF.

Eksempel 1 - Finn normalformen:

S(Brnavn, Navn, Etternavn, Kurskode, KursTittel, Karakter)

FDer:

- Brnavn, Kurskode $\rightarrow$ Karakter
- Brnavn $\rightarrow$ Navn
- Brnavn $\rightarrow$ Etternavn
- Kurskode $\rightarrow$ KursTittel

Kandidatnøkkel: {Brnavn, Kurskode}.

1. Brnavn, Kurskode er en supernøkkel, så BCNF så langt. (Dette er FD nummer 1)
2. Brnavn ikke supernøkkel, brudd på BCNF. (Dette er FD nummer 2). Dermed går vi til punkt nummer 2 i algoritmen over:
Er Navn et nøkkelattribut? Nei, dermed har vi et brudd på 3NF, og går til punkt nummer 3 i algoritmen:
Er Brnavn en del av en kandidatnøkkel? - Ja, dermed er det brudd på 2NF og skjemaet er på 1NF.
Dermed er relasjonen S på normalformen 1NF.

Oblig - Finn normalformen:

AgenterPåOppdrag(agentID, navn, initialer, født, oppdragsNavn, varighet, lokasjon).

FDer:

- agentId $\rightarrow$ navn, født
- navn, født $\rightarrow$ agentId
- navn $\rightarrow$ initialer
- oppdragsNavn $\rightarrow$ varighet, lokasjon

kandidatnøkler: {agentId, oppdragsNavn} og {navn, født, oppdragsNavn}.

FD 1: agentId er ikke en supernøkkel, altså er det brudd på BCNF. Går til steg nummer 2.

FD 1: navn og født er nøkkelattributter, går til neste FD.

FD 2: navn, født er ikke en supernøkkel, altså er det brudd på BCNF. Går til steg nummer 2.

FD 2: agentId er nøkkelattributter, går til neste FD.

FD 3: navn er ikke en supernøkkel, altså er det brudd på BCNF. Går til steg nummer 2.

FD 3: initialer er ikke et nøkkelattributt, går til steg nummer 3.

FD 3: navn er en del av en kandidatnøkkel, dermed er det brudd på 2NF.

Relasjonen AgenterPåOppdrag er på 1NF.

Tapsfri dekomposisjon:

Hvordan oppnå BCNF?

- Hvordan lage et nytt skjema på BCNF som inneholder den samme informasjonen?
- Problemet er, grovt sett, at data som ikke hører sammen i samme tabell.
- Kan løses ved å DEKOMPONERE tabelen til mindre tabeller.
- Kan ikke dekomponere som vi vil - dekomposisjonen må være tapsfri. Altså må all informasjonen som vi startet med være til stede etter dekomponeringen.

Ikke tapsfri dekomponering:

Opprinnelig tabell: Ansatte(AvdID, AvdNavn, AnsattID, Navn, Etternavn).

Dekomponert til:

- Avdeling(AvdID, AvdNavn)
- Ansatt(AnsattID, Navn, Etternavn)

Denne dekomponeringen er ikke tapsfri fordi alle attributter er med, men vi har mistet noe viktig, nemlig FORHOLDET MELLOM ansatt og avdeling!!!!

For at den skulle vært tapsfri så måtte vi kanskje hatt noe slikt også:

- AnsattV(AvdID, AnsattID)

Tapsfri dekomponering, eksempel

Brnavn	Navn	Etternavn	Adresse	Kurskode	Tittel	Beskrivelse	AntSP	Kara
evgenit	Evgenij	Thorstensen	Addr1	IN2090	Databaser	EnBeskr...	10	B
peternl	Petter	Nilsen	Addr2	IN2090	Databaser	EnBeskr...	10	A
evgenit	Evgenij	Thorstensøn	Addr1	IN2080	Beregn...	Descr...	10	A
leifhka	Leif H.	Karlsen	Addr3	IN2090	Databaser	EnBeskr...	10	B
leifhka	Leif H.	Karlsen	Addr3	IN3110	Program...	EnBeskr2...	5	C

Student			
Brnavn	Navn	Etternavn	Adresse
evgenit	Evgenij	Thorstensen	Addr1
peternl	Petter	Nilsen	Addr2
leifhka	Leif H.	Karlsen	Addr3

Kurs			
Kurskode	Tittel	Beskrivelse	AntSP
IN2090	Databaser	EnBeskr...	10
IN2080	Beregn...	Descr...	10
IN3110	Program...	EnBeskr2...	5

Karakter		
Brnavn	Kurskode	Kara
evgenit	IN2090	B
peternl	IN2090	A
evgenit	IN2080	B
leifhka	IN2090	B
leifhka	IN3110	C

Alle attributter er med, og naturlig join gir opprinnelig tabell.

Hvordan garantere tapsfri dekomponering?

- Fagins teorem:
En dekomponering av $R(X, Y, Z)$ til $S1(X, Y)$, $S2(X, Z)$ er tapsfri hvis og bare hvis $X \rightarrow Y$
- Med andre ord, vi kan skille ut noen attributter og det de alle er avhengig av.
- Dette gir opphav til dekomponeringsalgoritmen for BCNF.

TAPSFRI DEKOMPONERING TIL BCNF:

Tapsfri dekomponering av $R(X)$ med FDer F:

1. Beregn nøklene til R (fra F)
2. Splitt alle FDer i F slik at det kun er ett attributt på høyresiden av hver FD
(f.eks $A, B \rightarrow C, D$ blir $A, B \rightarrow C$ og $A, B \rightarrow D$)
3. Sjekk om R bryter med BCNF (den algoritmen som vi brukte i forrige oppgave)!!!!!!
 - 3.1 Hvis R ikke bryter med BCNF (altså er på BCNF), stopp og returner R .
 - 3.2 Hvis R bryter med BCNF:
 - 3.2.1 Finn én FD $Y \rightarrow A$ (element) F som bryter med BCNF
 - 3.2.2 Beregn Y^+ med hensyn på FDene i F
 - 3.2.3 Dekomponer R til $S1(Y^+)$ og $S2(Y, X/Y^+)$
 - 3.2.4 Fortsett rekursivt over $S1$
(med FDene som kun inneholder attributter fra $S1$ (altså Y^+))

3.2.5 Fortsett rekursivt over S2

(med FDene som kun inneholder attributter fra S2 (altså $Y, X/Y^{+}$))

Oblig oppgave:

$R(A, B, C, D, E, F)$

FDer:

1. $A \rightarrow BC$
2. $BC \rightarrow A$
3. $D \rightarrow E$
4. $AD \rightarrow F$
5. $E \rightarrow F$

1. Beregner nøklene til R: $\{A, D\}$ og $\{B, C, D\}$
2. $A \rightarrow BC$, ved å bruke algoritmen for å finne normalformen får vi at det er brudd på BCNF, men ikke på 3NF siden BC er et nøkkelattributt.
3. Finner tillukningen til venstresiden:
 $A^{+} = A, B, C$
 $S1(A, B, C)$. S1 har FDene 1. og 2.
 $S2(A, D, E, F)$. S2 har FDene 3., 4. og 5.
4. Fortsetter med $S2(A, D, E, F)$
FDer:
 $D \rightarrow E$
 $AD \rightarrow F$
 $E \rightarrow F$
Kandidatnøklen er: $\{A, D\}$.
 $D \rightarrow E$, brudd på BCNF
E er ikke et nøkkelattributt, og bryter med 3NF
D er del av en kandidatnøkkel.
 $D^{+} = D, E$
 $S21(D, E)$ og $S22(D, A, F)$
5. Dermed kan $R(A, B, C, D, E, F)$ dekomponeres til $S1(A, B, C)$, $S21(D, E)$ og $S22(D, A, F)$.

Eksempel 1:

La $R(A, B, C)$

FDer:

- $AB \rightarrow C$
- $C \rightarrow A$

1. Beregner nøklene til R : $\{A, B\}$ og $\{B, C\}$.
2. Bruker algoritmen for å sjekke BCNF (to over!)
 $AB \rightarrow C$ er ikke brudd på BCNF, siden AB er en supernøkkel.
 $C \rightarrow A$ er brudd på BCNF, men ikke på 3NF, siden A er et nøkkelattributit.
3. Vi skal finne tillukningen til venstresiden:
3.2 Beregn $C^+ = CA$
3.3 Dekomponerer R til $S1(C, A)$ og $S2(C, B)$. (Tar med B siden det er den eneste som ikke er med i tillukningen til C).
3.4 Siden $AB \rightarrow C$, så gjelder ikke denne for $S1$ siden B er med. $C \rightarrow A$ gjelder for $S1$ siden C og A er med i $S1$
3.5 $S2$ inneholder A i den første og andre FDen. Dermed har den ingen FDer.
Kun én FD som holder for $S1(C \rightarrow A)$ og bryter ikke med BCNF og ingen FDer for $S2$, dermed siden $S1$ ikke bryter med BCNF og $S2$ ikke har noen FDer så bryter hverken $S1$ eller $S2$ med BCNF
4. Dermed kan $R(A, B, C)$ dekomponeres til $S1(C, A)$ og $S2(C, B)$

Eksempel 2:

$S(\text{Brnavn}, \text{Navn}, \text{Etternavn}, \text{Kurskode}, \text{KursTittel}, \text{Karakter})$

FDer:

1. $\text{Brnavn}, \text{Kurskode} \rightarrow \text{Karakter}$
2. $\text{Brnavn} \rightarrow \text{Navn}$
3. $\text{Brnavn} \rightarrow \text{Etternavn}$
4. $\text{Kurskode} \rightarrow \text{KursTittel}$

Kandidatnøkkel: $\{\text{Brnavn}, \text{Kurskode}\}$.

1. Brnavn $\rightarrow$ Navn, bryter med BCNF

3.2 Beregner $\text{Brnavn}^+ = \{\text{Brnavn}, \text{Navn}, \text{Etternavn}\}$.

3.3 Får da $S1(\text{Brnavn}, \text{navn}, \text{Etternavn})$ og $S2(\text{Brnavn}, \text{Kurskode}, \text{Kurstittel}, \text{Karakter})$

3.4 $S1$ har FDene 2. og 3. (altså der alt det de inneholder er attributter fra $S1$. 1. og 4. inneholder Kurskode), men ingen av disse bryter med BCNF.

3.5 $S2$ har FDene 1. og 4.

Fortsetter med $S2$:

$S2(\text{Brnavn}, \text{Kurskode}, \text{Kurstittel}, \text{Karakter})$

FDer:

1. Kurskode $\rightarrow$ Kurstittel
2. Brnavn, Kurskode $\rightarrow$ Karakter
 - Kandidatnøkkel: $\{\text{Brnavn}, \text{Kurskode}\}$
 - Kurskode $\rightarrow$ Kurstittel bryter med BCNF
 - $\text{Kurskode}^+ = \text{Kurskode}, \text{Kurstittel}$
 - Får $S21(\text{Kurskode}, \text{Kurstittel})$ og $S22(\text{Kurskode}, \text{Brnavn}, \text{Karakter})$
 - $S21$ har kun første FD som ikke bryter med BCNF
 - $S21$ har kun andre FD som ikke bryter med BCNF

$S(\text{Brnavn}, \text{Navn}, \text{Etternavn}, \text{Kurskode}, \text{Kurstittel}, \text{Karakter})$

FDer:

1. Brnavn, Kurskode $\rightarrow$ Karakter
2. Brnavn $\rightarrow$ Navn
3. Brnavn $\rightarrow$ Etternavn
4. Kurskode $\rightarrow$ Kurstittel

Dekomponeres altså til:

$S1(\text{Brnavn}, \text{Navn}, \text{Etternavn})$

$S21(\text{Kurskode}, \text{Kurstittel})$

$S22(\text{Kurskode}, \text{Brnavn}, \text{Karakter})$

Dekomponering i praksis:

- Algoritmen gir oss den riktige strukturen på tabellene
- Må etterpå gi tabellene meningsfulle navn og sette skranker på kolonner
- Dersomman startet med et skjema som inneholdt data må disse flyttes over
- Ett en tabell $R(X)$ som dekomponeres til $S_1(Y_1), \dots, S_n(Y_n)$ kan dette gjøres ved å kjøre følgende for hver S_i :

```
INSERT INTO S_i  
SELECT DISTINCT Y_i  
FROM R;
```

Kandidatnøkkel:

N, A, F, G

Vi bruker N og A som kun forekommer på høyre side. I tillegg utvider vi med F, G og L. Dermed kan vi konstruere FGL som bestemmer K. Deretter kan vi konstruere NKL som bestemmer PB.

Nei vi har jo ikke K fra NK så vi kan ikke gjøre det. Kan K bli brukt i starten selv om den forekommer på begge sider?

Aldri Høyre: N, A

Aldri på venstre: P, B

Utvide med: K, L, F, G

Kandidatnøkler:

1. NKA. NK gir FG, A gir L, FGL gir K, NKL gir PB
2. NAFG. A gir L, FGL gir K, NKL gir PB, NK gir FG

AgenterPåOppdrag er på normalformen 1NF.

2NF - en tabell BRYTER 2NF hvis det finnes et ikke-nøkkelattributt A som ER AVHENGIG av en delmengde av en kandidatnøkkel.

Kandidatnøkkel 1 ({agentID, oppdragsNavn}):

- Initialer (ikke-nøkkelattributt) er avhengig av navn (delmengde av kandidatnøkkel)
- Varighet (ikke-nøkkelattributt) er avhengig av oppdragsNavn (delmengde av kandidatnøkkel)
- Lokasjon (ikke-nøkkelattributt) er avhengig av oppdragsNavn (delmengde av kandidatnøkkel)

Ikke nøkkelattributter: inititaler, varighet, lokasjon

{agentID, oppdragsNavn} _____ navn og født, er kun avhengig av agentID og ikke oppdragsNavn. Varighet og lokasjon er kun avhengig av oppdragsNavn. Dermed bryter det med 2NF.

{navn, født, oppdragsnavn} _____ agentID er kun avhengig av navn og født ikke oppdragsNavn. Varighet og lokasjon er kun avhengige av oppdragsNavn og ikke navn og født. Dermed bryter det med 2NF.

Den bryter med 3NF på den første kandidatnøkkelen, men ikke på den andre.